CipherMind

A local RAG assistant for Sysmon log investigation
with MITRE ATT&CK mapping and cited reports

Project Progress Report

Status as of 13 October 2026

Student	_____
Roll number	_____
Project guide	_____
Department / Institute	_____
Repository	github.com/SD0518/CipherMind

1. Executive summary

CipherMind takes a Windows Sysmon log, finds the suspicious events, maps them to MITRE ATT&CK techniques, and writes an incident report in which every finding cites the log line it came from. It runs fully on a local machine: Mistral via Ollama, ChromaDB for retrieval, and MongoDB for storage. No cloud API is used.

All eight planned stages have been implemented and tested end to end. A web interface (Gradio) runs the full pipeline, shows the retrieved evidence, displays the cited report and keeps a history of past investigations. What remains is demo rehearsal, a backup demo video, and one known retrieval limitation that is documented in Section 7.

Stages built	ATT&CK techniques indexed	Test logs (dev + held-out)	Overall F1 (Mistral)	Invented IDs in final run
8 / 8	**679**	**2 + 1**	**0.53**	**0**

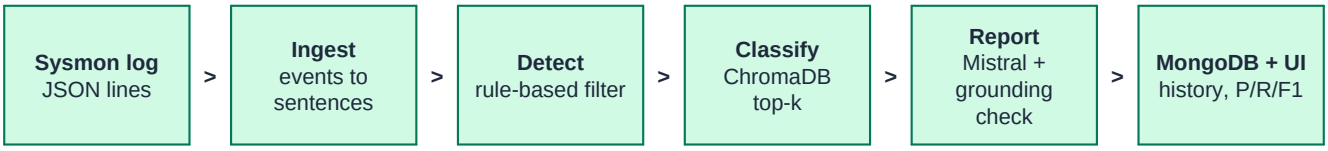
2. Stage status

Stages were built in a risk-first order (5, 4, 6, 3, 7, 8): the knowledge base was built first because retrieval quality decides whether everything after it can work.

#	Stage	Status	Evidence (test script and result)
1	Environment setup (Ollama Mistral, MongoDB, Gradio)	Done	check_setup.py: Mistral answers from Python; MongoDB reachable
2	Data: attack logs, ground truth, ATT&CK STIX	Done	check_data.py: 3 logs load; 6 ground-truth IDs exist in ATT&CK
3	MongoDB layer (logs, investigations, evaluations)	Done	check_db.py: write/read in all 3 collections; no duplicates on re-ingest
4	Sysmon ingest and normalization	Done	check_ingest.py: clean sentences, IDs masked; all dev techniques retrievable
5	ATT&CK knowledge base in ChromaDB	Done	check_kb.py: 5/5 test queries find the expected technique in top 5
6	Detect + retrieve (LangGraph flow)	Done	check_pipeline.py: ingest -> detect -> classify -> report runs on all logs
7	Cited Mistral report + evaluation	Done	evaluate.py: report, stored investigation and P/R/F1 per log
8	Gradio UI and demo preparation	Code done	App tested end to end; demo rehearsal and backup video pending

3. System architecture

The pipeline is a LangGraph flow of four nodes. Each node adds to a shared state; results are saved to MongoDB and shown in the UI.



Component	Technology	Role
LLM	Mistral 7B via Ollama (local)	Chooses techniques from retrieved candidates; writes the summary
Embeddings	sentence-transformers all-MiniLM-L6-v2	Embeds ATT&CK techniques and log sentences
Vector store	ChromaDB (persistent, cosine)	679 active ATT&CK techniques
Orchestration	LangGraph + LangChain	ingest -> detect -> classify -> report
Database	MongoDB (localhost)	logs, investigations, evaluations
Interface	Gradio 5 (dark theme)	Analyze, Retrieval, Report and History tabs

4. Work completed

Data (Stage 2)

- Three Mordor (OTRF Security-Datasets) logs from Atomic Red Team simulations: a BITS download (bitsadmin), a Windows service hijack (Fax service set to run PowerShell), and an mshta/JavaScript attack.
- Ground truth recorded per log in *data/ground_truth.json*; logs marked **dev** (used while building) or **test** (held out, never tuned on).

Knowledge base (Stage 5)

- 679 active techniques loaded from the ATT&CK STIX file (revoked/deprecated skipped); sub-techniques carry their parent name.
- Each technique embeds its name, description and detection text; one-time build takes about 21 s.

Ingest and normalization (Stage 4)

- Keeps useful Sysmon event types (process, network, file, registry, DNS) and turns each into a plain-English sentence; identical sentences are merged with all their source IDs.
- Every sentence keeps its source event ID (*file#Lline*) for traceability.
- ATT&CK IDs written inside the logs (e.g. 'T1543.003 Test') are masked so retrieval cannot read the answer; encoded PowerShell is decoded.

Detect and retrieve (Stage 6)

- Explainable rules flag suspicious behaviour (abused Windows tools, unusual parents, service/autostart registry changes, public IP connections). Rules never name a technique.
- Noise reduction: 38 to 3, 63 to 6 and 111 to 4 events across the three logs.

MongoDB layer (Stage 3)

- Raw events with their sentences, full investigations and evaluation scores are stored.
- If MongoDB is down, analysis still runs and only shows a warning (tested).

Cited report and evaluation (Stage 7)

- Per event, Mistral sees only the top-3 retrieved techniques and answers in JSON; code then removes any cited ID that was not retrieved for that event (grounding check).
- A summary call writes the attack narrative and recommended actions; *evaluate.py* scores every log against a retrieval-only baseline and saves the results.

User interface (Stage 8)

- Gradio app with step-by-step progress, stat cards, technique cards, report preview, retrieval table, downloadable Markdown report and MongoDB-backed history.
- A 'retrieval-only' mode gives a fast report without Mistral (demo fallback).

5. Results

Precision, recall and F1 compare the unique technique IDs predicted for each log with its ground truth, micro-averaged over logs. The retrieval-only rows show what the system achieves without the LLM.

Data	Method	Precision	Recall	F1
Dev (2 logs)	Retrieval top-1	0.40	0.50	0.44
Dev (2 logs)	Retrieval top-3	0.23	1.00	0.38
Dev (2 logs)	**Mistral + retrieval**	**0.50**	**0.75**	**0.60**
Test (held out)	Retrieval top-1	0.33	0.50	0.40
Test (held out)	Retrieval top-3	0.09	0.50	0.15
Test (held out)	**Mistral + retrieval**	**0.33**	**0.50**	**0.40**
All 3 logs	**Mistral + retrieval**	**0.44**	**0.67**	**0.53**

Per-log outcome (final run)

Log	Split	Expected	Found	Missed	Mistral time
bitsadmin_download	dev	T1197, T1105	T1197	T1105	about 60 s
service_mod_fax	dev	T1543.003, T1059.001	both	-	about 120 s
mshta_javascript	test	T1218.005, T1059.007	T1218.005 (rank 1, similarity 0.67)	T1059.007	about 85 s

- **Grounding works:** 0 invented technique IDs reached the final reports. Testing exposed a real bug (Mistral answering 'T1197 BITS Jobs' instead of 'T1197'), which was fixed by extracting the ID before the check.
- **Honest held-out result:** on the log nothing was tuned on, the main technique (Mshta, T1218.005) was found at rank 1, but Mistral did not improve on retrieval alone (F1 0.40 both).
- **Speed:** about 1-2 minutes per log on an RTX 2050 (4 GB); Mistral runs partly on GPU, partly on CPU.

6. Testing and verification

Each stage has its own test script, and all of them pass on the current code:

Script	What it verifies
check_setup.py	Mistral answers from Python; MongoDB reachable
check_data.py	Logs load; every ground-truth ID exists in ATT&CK and is not deprecated
check_kb.py	Knowledge base returns the expected technique for 5 test queries
check_ingest.py	Sentences are clean, no ATT&CK ID leaks, techniques retrievable
check_db.py	Read/write in all collections; re-ingest does not duplicate
check_pipeline.py	Full LangGraph flow and retrieval-only scores
evaluate.py	Precision/recall/F1 per log and split; diagnoses techniques missed by every method

Robustness checks also run: MongoDB offline (analysis continues), Ollama GPU failure (CPU-only fallback setting verified at about 8 tokens/s), and a before/after comparison confirming that UI changes did not alter the pipeline output.

7. Known limitations and issues

- **Missed JavaScript technique (T1059.007):** it ranks #7 for the mshta event because 'mshta' dominates the embedding, so it falls outside the 3 candidates Mistral sees. Smallest fix: show more candidates (top-7), tested on dev logs only.
- **Network/DNS false positives:** Mistral sometimes maps plain connections or DNS lookups to weak matches (e.g. 'Exfiltration Over Other Network Medium'). A prompt rule did not fix it.
- **Small evaluation set:** three logs (six techniques) are enough for a demo but not for strong statistical claims.
- **Debatable labels:** T1569.002 (Service Execution) on the Fax log is arguably correct but was not added to the ground truth after seeing results, to keep the evaluation honest.
- **Local GPU:** one transient Ollama CUDA error occurred; restarting Ollama fixed it, and a CPU-only setting exists as a fallback.

8. Next steps

Priority	Task
1	Rehearse the 5-minute demo; pre-run the demo logs so History has results
2	Record a backup demo video
3	Address the T1059.007 miss (top-7 candidates), evaluated on dev logs; report the held-out score separately
4	Add more untuned logs to strengthen the evaluation
5	Small code cleanups (shared log loader, shared finding builder) and faster analysis by skipping near-duplicate events

9. Credits

- **Cyber-Agent** (github.com/minhan6559/Cyber-Agent): base project; its original code is kept for reference. CipherMind's pipeline is new code and does not import it.
- **MITRE ATT&CK**: technique data. **OTRF Security-Datasets (Mordor)** and **Atomic Red Team**: attack logs.